API FileFlow — Guide d'utilisation

Titre	Documentation de l'API REST — Plateforme FileFlow
Version	1.1
Date	27 octobre 2026
Statut	Validé
Propriétaire	Driss OUAAZIZ — Software Architect

Ce document décrit l'usage pratique de l'API REST de FileFlow. Le contrat complet et faisant foi est la spécification `openapi.yaml` (OpenAPI 3.1).

- **Base** : toutes les routes sont préfixées par `/api` et servies sur la même origine que l'application web.
- **Format** : requêtes et réponses en JSON (`application/json`), sauf le téléversement multipart, le téléchargement binaire et les pages HTML publiques.
- **Version** : 1.1.0.

1. Authentification

L'API accepte deux mécanismes d'authentification, équivalents côté serveur :

MÉCANISME	USAGE RECOMMANDÉ	DÉTAILS
Cookie de session	Application web (navigateur)	Cookie `fileflow_token` déposé à la connexion. `HttpOnly`, `SameSite=Strict`, `Secure` (désactivable en HTTP local via `COOKIE_SECURE=false`).
Jeton Bearer	Clients hors navigateur, scripts	En-tête `Authorization: Bearer <jwt>` (le JWT est la valeur du cookie).

Depuis un navigateur, les appels doivent inclure les cookies :

```
fetch('/api/files?path=/', { credentials: 'include' })
```

Le jeton porte une **version** (`token_version`). Un changement de mot de passe, une déconnexion globale, une révocation par un administrateur, une modification des droits ou l'activation/désactivation de la 2FA invalide immédiatement tous les jetons antérieurs. La version initiale d'un compte est aléatoire : un jeton émis pour un compte supprimé ne redevient pas valide si un compte homonyme est recréé.

Déconnexion

`POST /auth/logout` ne requiert pas de jeton valide (absent, expiré ou révoqué accepté) et efface **toujours** les deux cookies `fileflow_token` et `fileflow_2fa_pending`. `POST /auth/logout-all` (authentifié) révoque en plus toutes les sessions de l'utilisateur sur tous les appareils.

Connexion

```
curl -i -X POST https://<hôte>/api/auth/login \
  -H 'Content-Type: application/json' \
  -c cookies.txt \
  -d '{"username":"admin","password":"<motdepasse>"}'
```

- Sans 2FA : la réponse `200` porte le profil (`UserInfo`) et dépose `fileflow_token`.

- Avec 2FA : la réponse `200` contient `{ "requires_2fa": true }` et dépose un cookie temporaire `fileflow_2fa_pending`.

Flux de double authentification (TOTP)

Le cookie `fileflow_2fa_pending` est **valable 5 minutes**. L'appelant vérifie ensuite le code à 6 chiffres (ou un code de récupération) :

```
curl -i -X POST https://<hôte>/api/auth/2fa/verify \
  -H 'Content-Type: application/json' \
  -b cookies.txt -c cookies.txt \
  -d '{"code":"123456"}'
```

En cas de succès, le cookie temporaire est remplacé par le cookie de session complet. L'activation de la 2FA (`/auth/2fa/setup` → `/auth/2fa/confirm`) renvoie les **codes de récupération à usage unique** uniquement au moment de la confirmation — ils ne sont affichés qu'une fois. `setup` et `confirm` sont refusés (`400`) si la 2FA est déjà active. La confirmation fait tourner la session : les autres sessions sont révoquées et un nouveau cookie est déposé.

Modifier une 2FA active exige le mot de passe **et** un second facteur (code TOTP courant ou code de récupération) :

```
curl -i -X POST https://<hôte>/api/auth/2fa/disable -b cookies.txt -c cookies.txt \
  -H 'Content-Type: application/json' \
  -d '{"password":"<motdepasse>","code":"123456"}'
```

Le même corps `{ password, code }` est exigé par `/auth/2fa/recovery-codes/regenerate`. La désactivation fait également tourner la session (nouveau cookie, autres sessions révoquées).

---

2. Conventions d'erreur

Toutes les erreurs suivent une enveloppe unique :

```
{ "error": "Message lisible décrivant le problème" }
```

CODE	SIGNIFICATION
`400`	Requête invalide (paramètre manquant, quota dépassé, type de fichier interdit, modification du dernier administrateur actif…).
`401`	Authentification requise ou jeton invalide/révoqué.
`403`	Droits insuffisants (contrôle d'accès).
`404`	Ressource introuvable (ou lien public expiré, plafond atteint, créateur désactivé ou privé d'accès — réponse volontairement uniforme).
`409`	Conflit (ex. offset de fragment inattendu lors d'un téléversement résumable).
`429`	Trop de tentatives — verrouillage anti-force-brute temporaire.
`500`	Erreur interne.

Les suppressions réussies répondent `204 No Content`.

---

3. Limitation de débit (anti-force-brute)

Une limitation s'applique **uniquement** aux étapes de vérification d'un secret. Toute clé verrouillée l'est **15 minutes** (réponse `429`) :

ROUTE	CLÉS COMPTÉES	SEUIL
`POST /auth/login`	adresse IP	5 échecs
	couple utilisateur + IP	5 échecs
	compte, toutes IP confondues	50 échecs (non remis à zéro par une connexion réussie)
`POST /auth/2fa/verify`, `/auth/2fa/verify-recovery`, `/auth/2fa/disable`, `/auth/2fa/recovery-codes/regenerate`	utilisateur	5 échecs
`POST /s/{token}`, `POST /r/{token}` (mot de passe d'un lien public)	lien **et** adresse IP	5 échecs

Un tiers ne peut donc pas bloquer un compte avec quelques mauvais mots de passe : il lui faudrait une cinquantaine d'échecs répartis sur au moins dix adresses IP. Les verrous sont tenus en mémoire et levés par un redémarrage du service. Les autres routes de l'API ne sont pas soumises à cette limitation. La résolution de l'IP côté serveur ignore les en-têtes de proxy (`X-Forwarded-For`, `X-Real-IP`) sauf si `TRUST_PROXY_HEADERS=true`.

---

4. En-têtes de sécurité

Toutes les réponses portent : `Content-Security-Policy` stricte (`script-src 'self'`), `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy: no-referrer`, et `Strict-Transport-Security` lorsque les cookies sécurisés sont actifs (TLS). Les fichiers de type « contenu actif » (HTML, SVG, XML…) sont servis en `text/plain` en prévisualisation pour neutraliser tout risque de script stocké.

---

5. Pagination et limites

- **Journal d'audit** (`GET /admin/audit`) : paramètres `limit` (défaut 500, maximum 5000) et `offset`, plus filtres `username`, `action`, `resource`, `from`, `to`.
- **Recherche** (`GET /files/search`) : terme d'au moins 2 caractères ; résultats bornés côté serveur.
- **Téléchargement groupé** : taille cumulée plafonnée ; nombre de constructions d'archives simultanées borné.
- **Fragments d'upload** : taille maximale de 32 Mio par fragment.
- **Dépôt public** (`POST /r/{token}`) : taille cumulée plafonnée par `PUBLIC_UPLOAD_MAX_MB` (100 Mio par défaut, jamais au-delà de `MAX_UPLOAD_MB`). Le champ `pwd` doit précéder les fichiers dans le corps multipart. Un dépôt n'écrase jamais un fichier existant : il reçoit un nom numéroté (`rapport (1).txt`).

---

6. Exemples

Lister un répertoire

```
curl -s 'https://<hôte>/api/files?path=/documents' -b cookies.txt
```

Téléversement résumable (fragmenté)

```
# 1) Initialisation
UPLOAD=$(curl -s -X POST https://<hôte>/api/uploads/init -b cookies.txt \
  -H 'Content-Type: application/json' \
  -d '{"path":"/documents","filename":"rapport.pdf","size":10485760}')
ID=$(echo "$UPLOAD" | python3 -c 'import sys,json;print(json.load(sys.stdin)
["upload_id"])')
```

```
# 2) Envoi d'un fragment à l'offset courant (répéter jusqu'à size)
curl -s -X PUT "https://<hôte>/api/uploads/chunk?upload_id=$ID&offset=0" \
  -b cookies.txt --data-binary @fragment_0.bin

# 3) Finalisation
curl -s -X POST https://<hôte>/api/uploads/finish -b cookies.txt \
  -H 'Content-Type: application/json' -d "{\"upload_id\":\"$ID\"}"
```

En cas de réponse `409` sur un fragment, relire l'offset attendu via `GET /uploads/status?upload_id=<id>` puis renvoyer le fragment à cet offset.

Créer un lien de partage (à usage unique, protégé par mot de passe)

```
curl -s -X POST https://<hôte>/api/shares -b cookies.txt \
  -H 'Content-Type: application/json' \
  -d
'{"file_path":"/documents/rapport.pdf","expires_in_secs":86400,"password":"secret","max_downloads":1}'
```

Le lien public est ensuite accessible sur `/api/s/<token>`. Le décompte des téléchargements est atomique ; un lien doté d'un plafond (`max_downloads`) ignore l'en-tête `Range` et sert toujours le fichier complet, afin qu'un lien à usage unique ne puisse pas être récupéré par morceaux. Le lien suit le fichier lors d'un renommage ou d'un déplacement et disparaît avec lui ; il répond `404` dès que son créateur est désactivé ou perd l'accès en lecture.

Créer une demande de téléversement

`POST /upload-requests` exige l'accès en écriture **propre** sur le dossier cible (rôle, permissions, groupes) : un accès obtenu via un partage interne ou un dossier partagé est refusé (`403`). Ces droits sont revérifiés à chaque dépôt.

---

7. Régénérer et visualiser la spécification

La spécification `openapi.yaml` est maintenue à la main et fait référence.

- **Visualiser** : ouvrir `openapi.yaml` dans un éditeur compatible OpenAPI, ou le servir via une interface Swagger UI / Redoc.
- **Valider** :

  ```
  python3 -c "import yaml; yaml.safe_load(open('openapi.yaml'))"   # syntaxe YAML
  npx @redocly/cli lint openapi.yaml                               # validation OpenAPI
  ```

- **Générer des clients** : `openapi-generator` ou `openapi-typescript` à partir du fichier.

Références

- **OpenAPI Specification** — https://www.openapis.org/ (contrat d'interface REST).
- **JSON Schema** — https://json-schema.org/ (schémas de données, utilisés par OpenAPI 3.1).
- **AsyncAPI** — *sans objet* : FileFlow n'expose pas d'API événementielle (pas de messagerie, de webhooks ni de flux temps réel).